

Introducción a Ethereum y SmartContracts

Mg. Francisco Gindre fgindre@lifa.info.unlp.edu.ar

Dr. Matías Urbietta

Facultad de Informática - Universidad Nacional de La Plata

O'REILLY®

Mastering Ethereum

BUILDING SMART CONTRACTS AND DAPPS



Andreas M. Antonopoulos
Dr. Gavin Wood

www.EBooksWorld.ir

Lectura sugerida

- Capitulo 1 Mastering Ethereum
- Beige Paper Ethereum

<https://github.com/chronaeon/beigepaper/blob/master/beigepaper.pdf>

- Proof of stake

<https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/>

Motivación

- 2013, Vitalik Buterin Whitepaper: propuso una extensión de bitcoin Turing-complete de propósito general con soporte a Smart contracts de forma rudimentaria

Componentes

- P2P network
- Reglas de consenso
- Transacciones
- **Maquina de estado**
- Estructuras de datos
- Economia: PoW → PoS
- Clientes

Los mismos que Bitcoin!!!
Pero en qué se diferencian?

Adopción

Ethereum en números

4k+

Projects built on Ethereum ⓘ

96M+

Accounts (wallets) with an
ETH balance ⓘ

53.3M+

Smart contracts on
Ethereum ⓘ

\$410B

Value secured on Ethereum ⓘ

\$3.5B

Creator earnings on
Ethereum in 2021 ⓘ

1,171 M

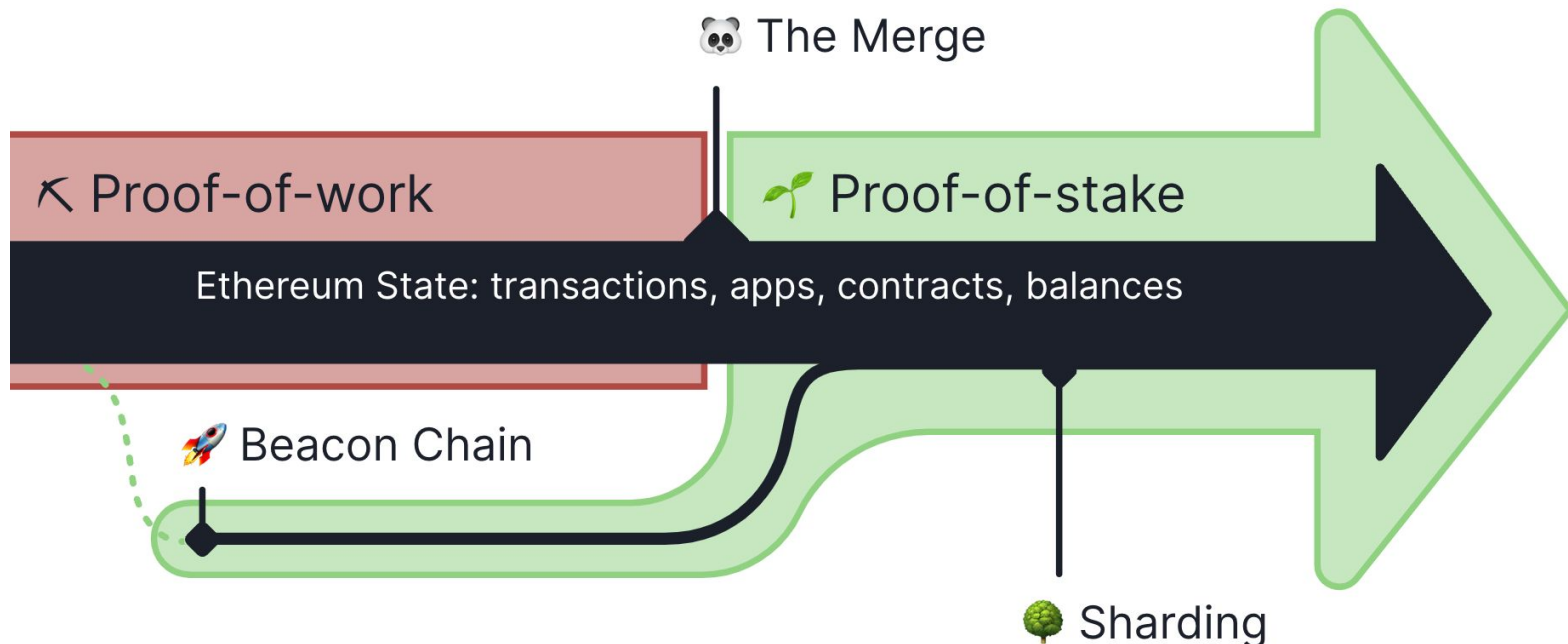
Number of transactions
today ⓘ

Proof of Stake (POS)

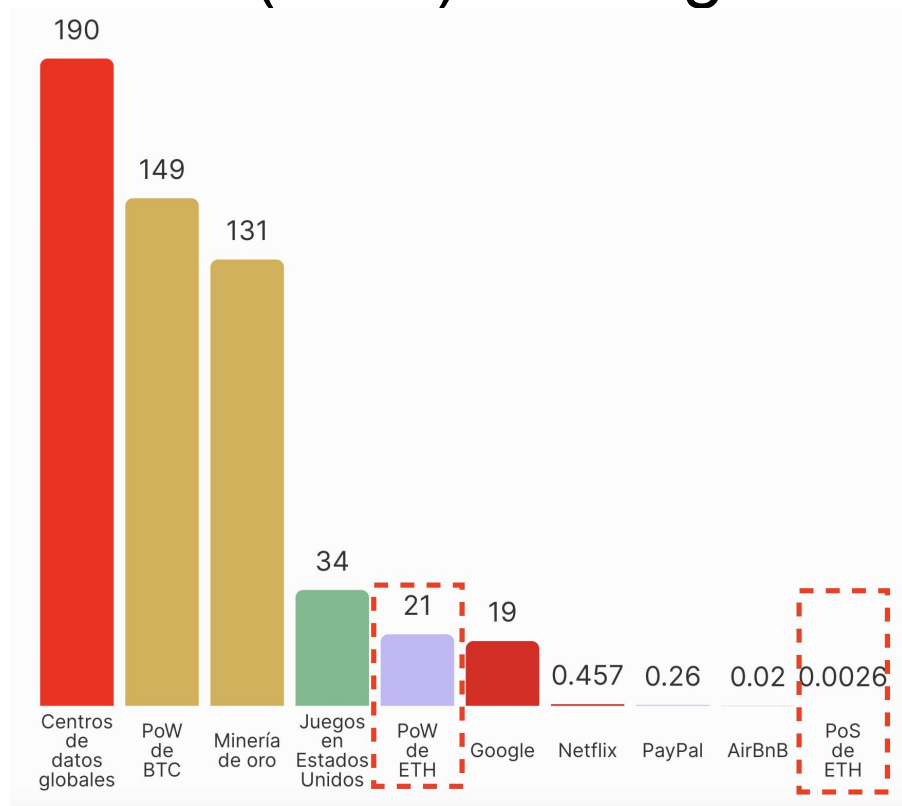
- Eficiencia energética
- Menores requisitos de hardware
- Menor riesgo de centralización
- Se requiere un seguro en ETH para incentivar la participación → 32 ETH para validador
- Penalidades económicas por mal comportamiento (evitan ataques del 51%)
 -

Ethereum - The merge/ fusión

- Inicia con POW y pasa a Proof of Stake
- La Fusión reduce el consumo de energía de Ethereum en un ~99.95%.



Proof of Stake (POS) - Energia



<https://ethereum.org/es/energy-consumption/>

- Gasper es el mecanismo de consenso para proof-of-stake
- Gasper es un mecanismo de consenso sobre una red proof-of-stake blockchain. Define como se reconoce o penaliza a los nodos.
-
-

<https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/gasper/>

<https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/gasper/>



Eth1



Eth2



**Execution
Layer**



**Consensus
Layer**

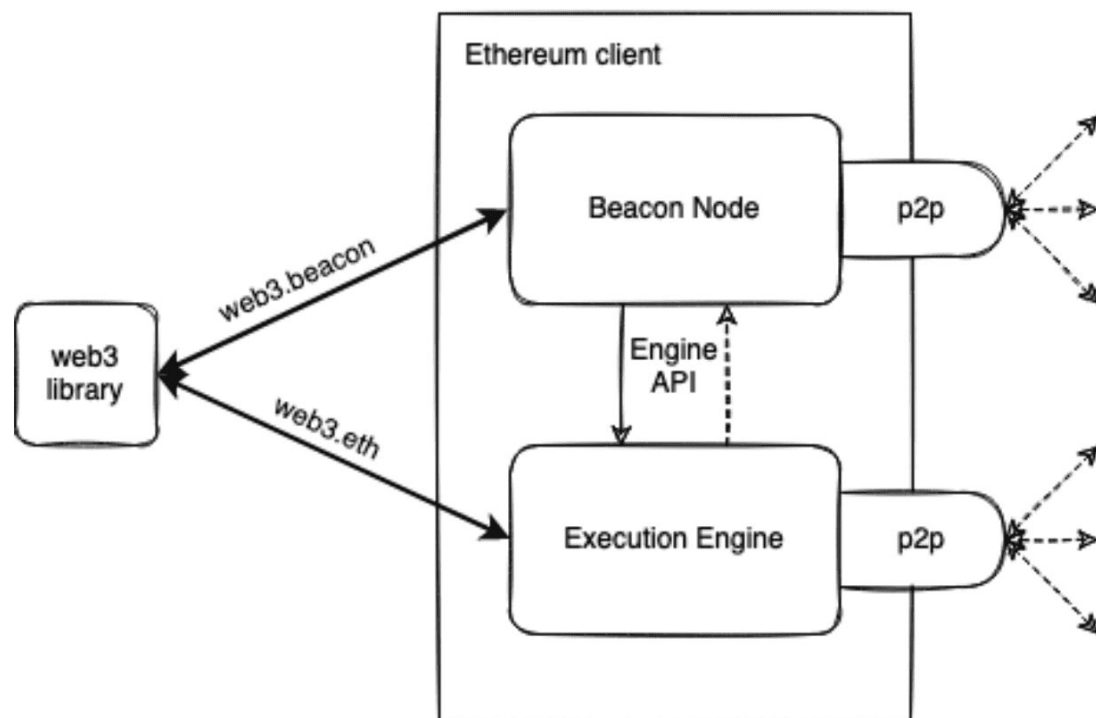


Ethereum

Nodos y clientes

- Execution Client, Execution Engine, EL client (o anteriormente el cliente Eth1).
 - Punto de entrada a la red y validación básica
 - Crea los mensajes a ejecutar (transacciones con datos de ejecución)
 - Escucha por transacciones, las ejecuta en la EVM y almacena su cambio de estado en la base de datos.
- Beacon Node, Consensus Layer (CL) client (o anteriormente el Eth2 client).
 - Lógica de sincronización
 - Verifica datos en base a las reglas de protocolo y mantiene la red segura.
 - Ejecuta las reglas de consenso Proof of Stake.

Cientes



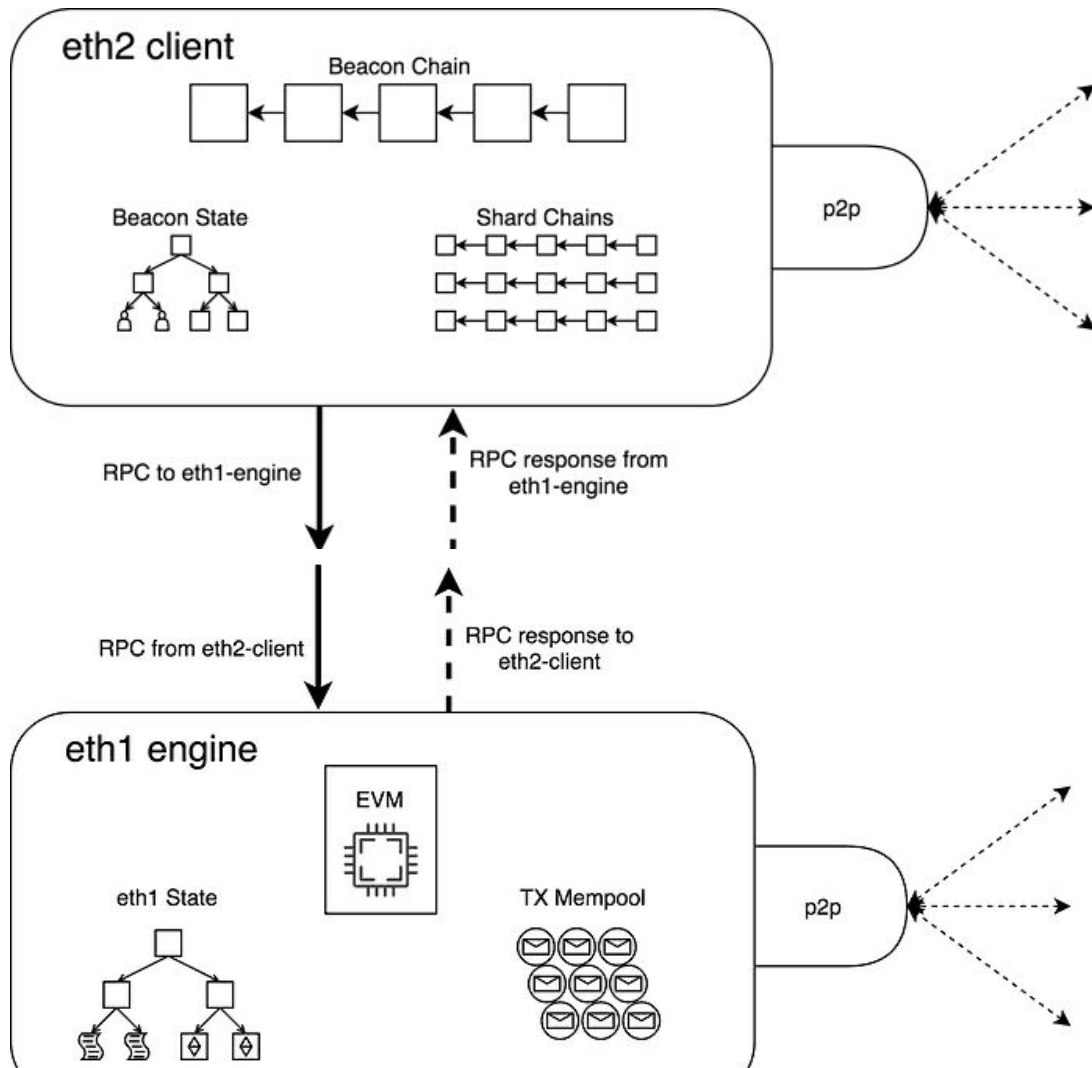
Tracking nodes

Ethereum Node Tracker: <https://etherscan.io/nodetracker#countries/ar/ar-all>

Ethereum Mainnet Statistics (Clientes): <https://ethernodes.org/>

Proof of Stake (POS)

CL,CC,ETH2



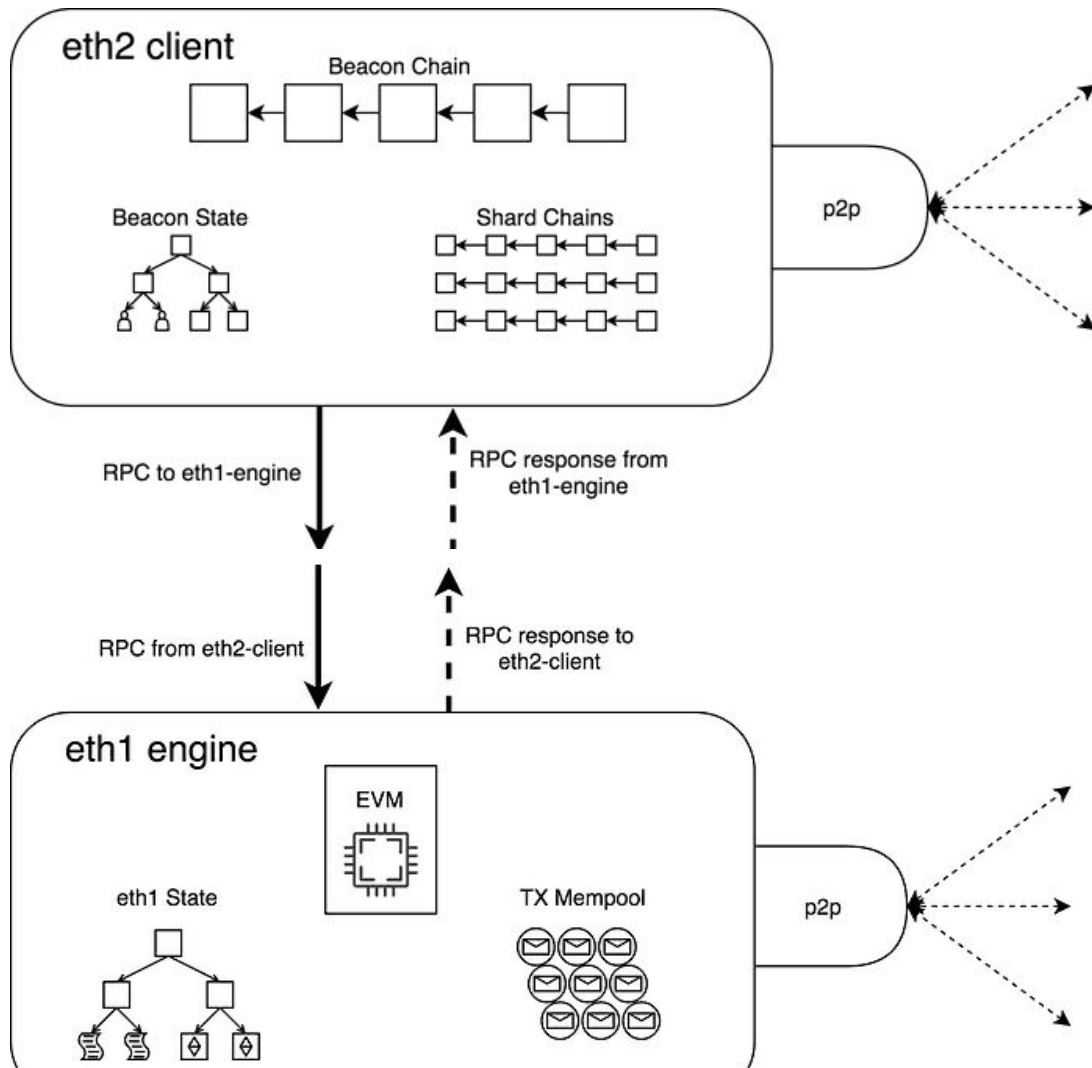
EL,EC,ETH1

Proof of Stake (POS) - Nueva transacción

- Se crea una transacción + base fee + tip definido en gas por una wallet
- La transacción se envía a un cliente de ejecución (EL) de Ethereum que verifica su validez (firmas, hashes, y **fondos**).
- Si es correcta se la agrega en el mempool, y se notifica a otros nodos en la capa de ejecución que hacen lo mismo.
- Un nodo es seleccionado aleatoriamente para construir un bloque usando RANDAO.

Proof of Stake (POS)

CL,CC,ETH2



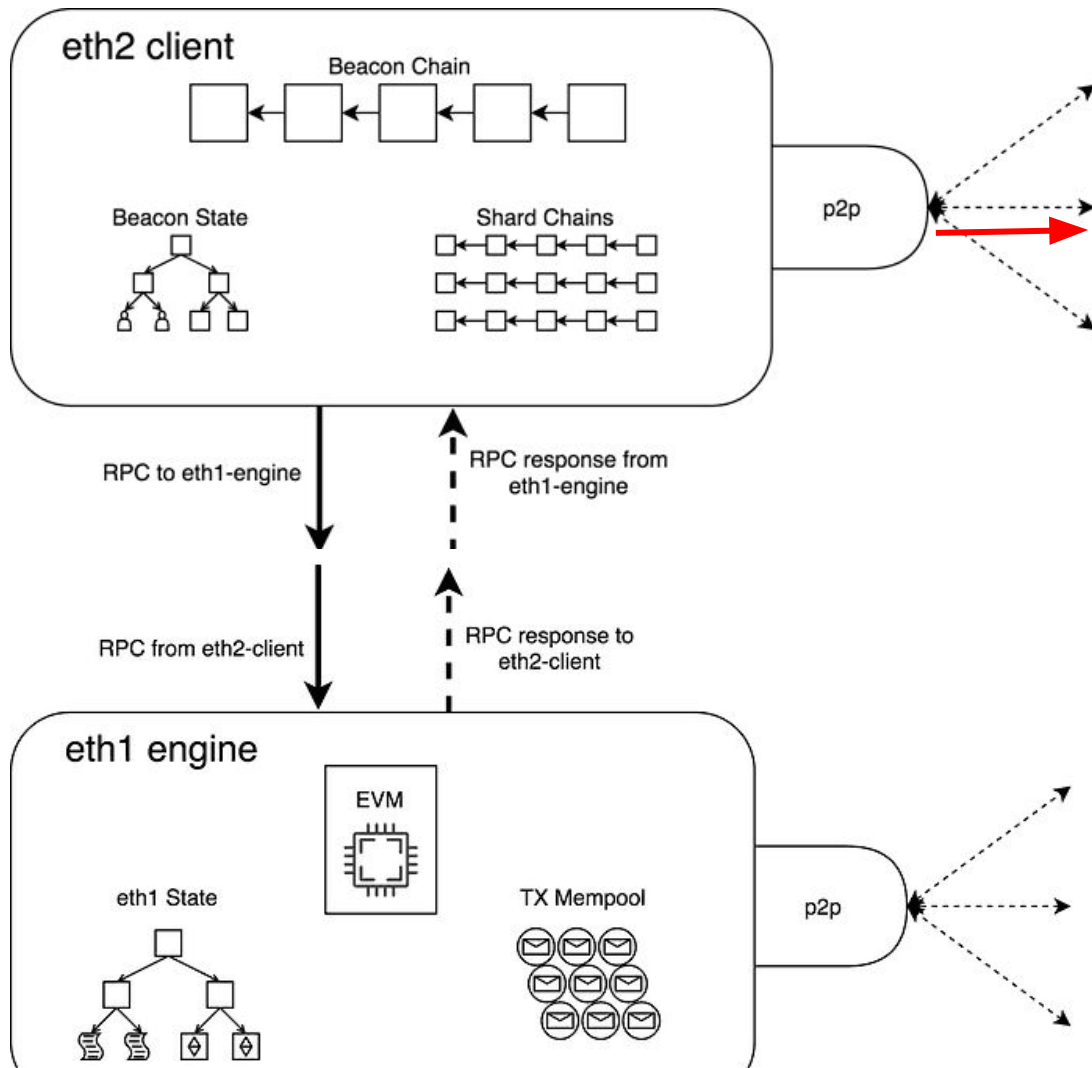
EL,EC,ETH1

Proof of Stake (POS) - Nodo como productor

- El cliente de Consenso (CL) recibe la notificación de que será el productor del próximo bloque.
- El CL dispara la instrucción de crear un bloque al cliente de ejecución (EL) LOCAL (local RPC)
- EL empaqueta transacciones disponibles en la mempool en un bloque, ejecuta la transacción y computa el hash.
- El CL toma el nuevo bloque (transacciones y hash) desde el EL lo agrega al beacon block (local RPC)
- El CL distribuye sobre la red el bloque donde otros nodos lo validan.

Proof of Stake (POS)

CL,CC,ETH2



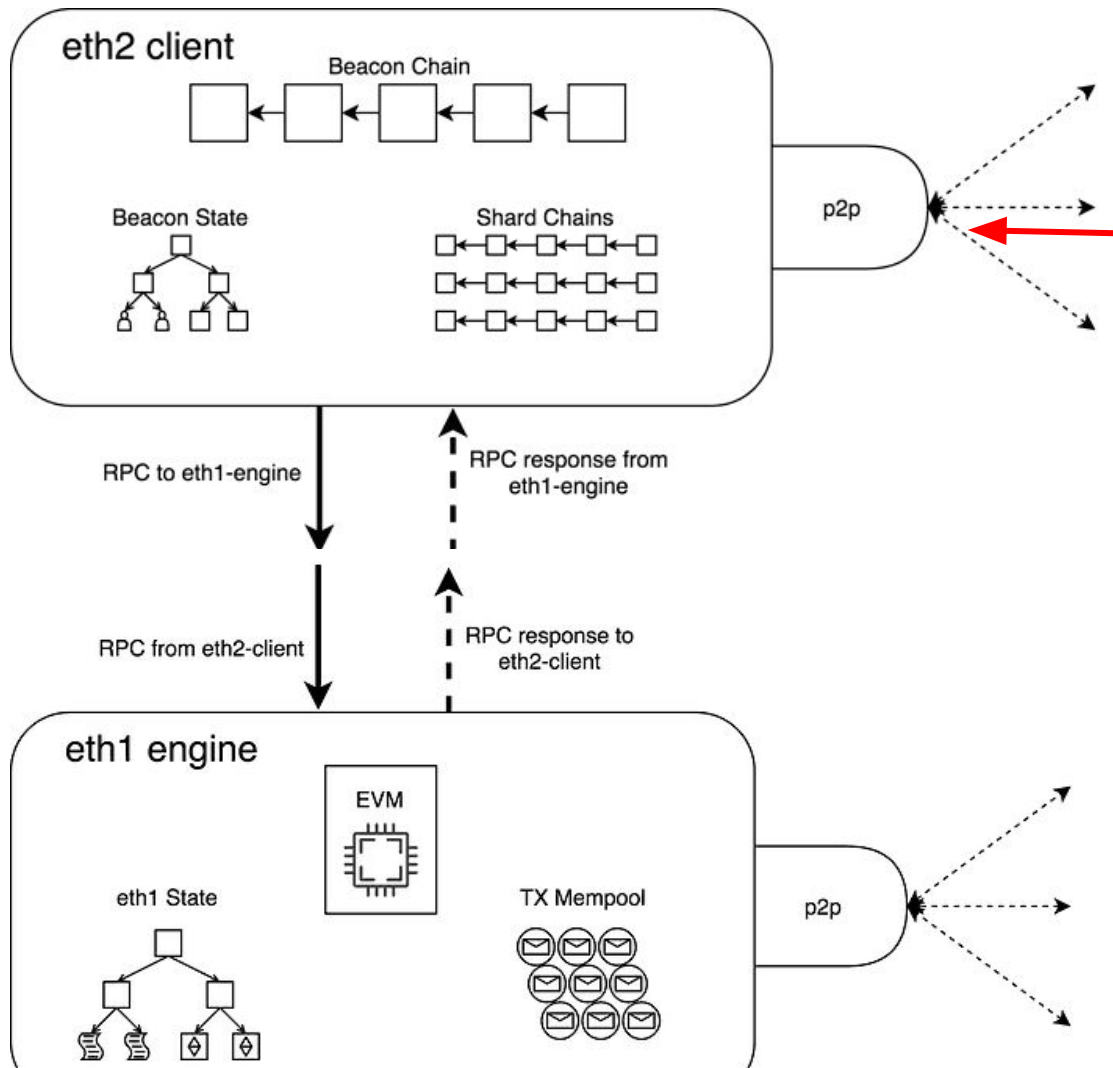
EL,EC,ETH1

Proof of Stake (POS) - Nodo NO productor

- Un CL recibe un bloque y lo pre-valida (remitente válido y datos correctos)
- El CL envía las transacciones del bloque a la EL
- La EL ejecuta las transacciones y validan el estado en el block header (hashes)
- Se retorna el resultado de validación al CL, y el bloque ahora confirmado.
- El CL agrega el bloque a la cabeza de su propio blockchain y lo confirma, distribuyendo este resultado a toda la red.

Proof of Stake (POS)

CL,CC,ETH2



EL,EC,ETH1

Proof of Stake (POS) - Finalización

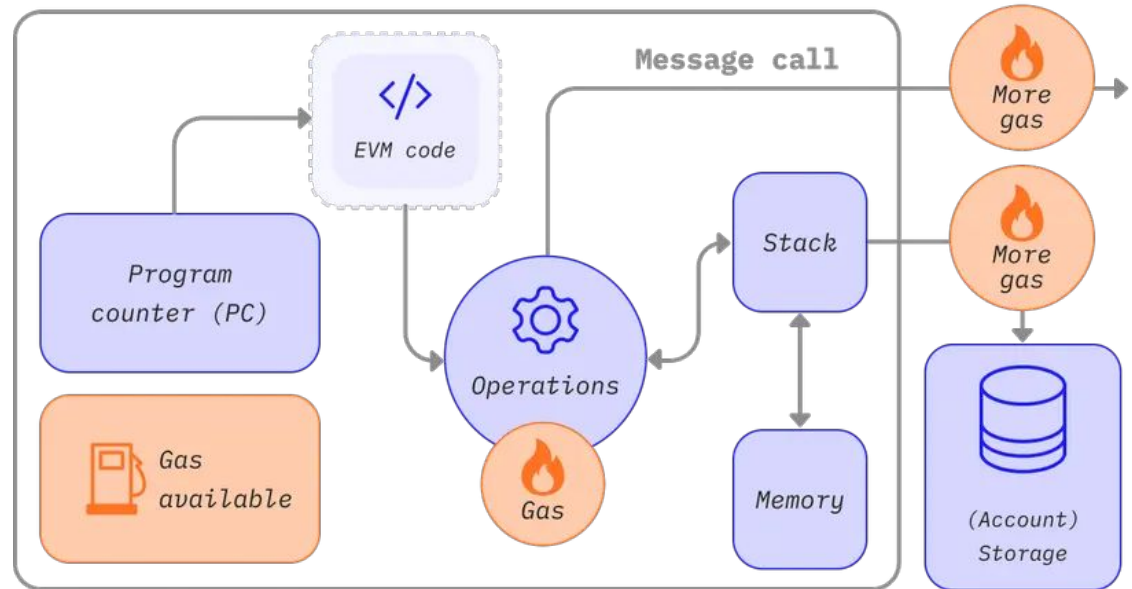
- Una transacción es considerada finalizada cuando no puede ser alterada por su alto costo en ETH.
- La red posee bloques de control (checkpoints) que son validados por los nodos.

Mc

Value (in wei)	Exponent	Common name	SI name
1	1	wei	Wei
1,000	10^3	Babbage	Kilowei or femtoether
1,000,000	10^6	Lovelace	Megawei or picoether
1,000,000,000	10^9	Shannon	Gigawei or nanoether
1,000,000,000,000	10^{12}	Szabo	Microether or micro
1,000,000,000,000,000	10^{15}	Finney	Milliether or milli
<i>1,000,000,000,000,000,000</i>	<i>10^{18}</i>	<i>Ether</i>	<i>Ether</i>
1,000,000,000,000,000,000,000	10^{21}	Grand	Kiloether
1,000,000,000,000,000,000,000,000	10^{24}		Megaether

GAS

- Es la unidad de medida del esfuerzo computacional
 - Previenen el SPAM y loops infinitos
- El fee es la cantidad de gas quemada multiplicado por unidad de gas



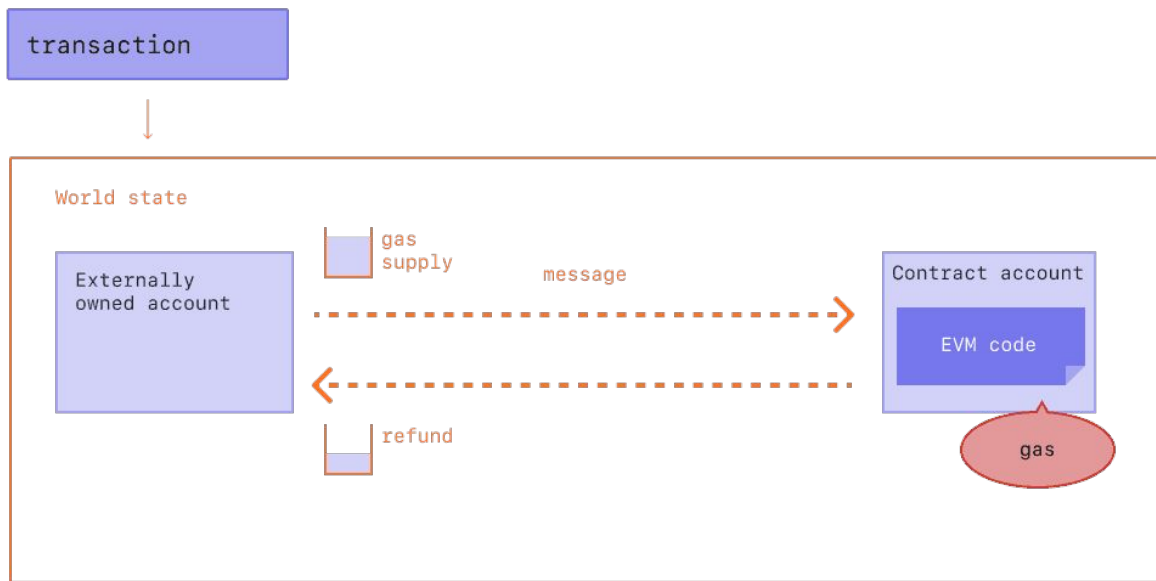
Transacciones



```
{  
  from: "0xEA674fdDe714fd979de3EdF0F56AA9716B898ec8",  
  to: "0xac03bb73b6a9e108530aff4df5077c2b3d481e5a",  
  gasLimit: "21000",  
  maxFeePerGas: "300",  
  maxPriorityFeePerGas: "10",  
  nonce: "0",  
  value: "100000000000"  
}
```

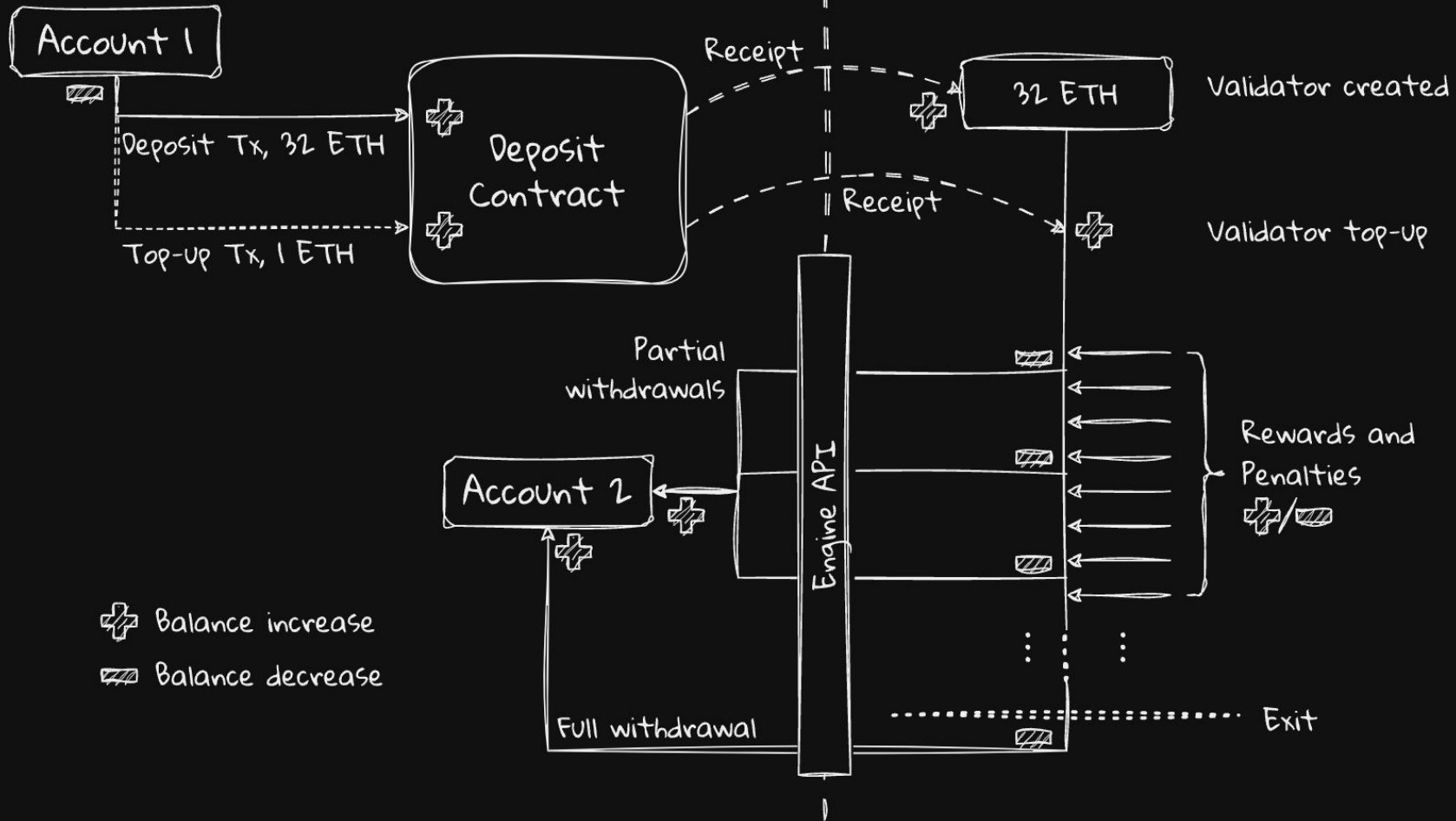
Gas y TIP

- Matias envia a Luis 1ETH con $\text{baseFeePerGas} = 190 \text{ gwei}$ y $\text{maxPriorityFeePerGas} = 10 \text{ gwei}$
 - $(190 + 10) * 21000 = 4,200,000 \text{ gwei}$ // o 0.0042 ETH
 - Message call

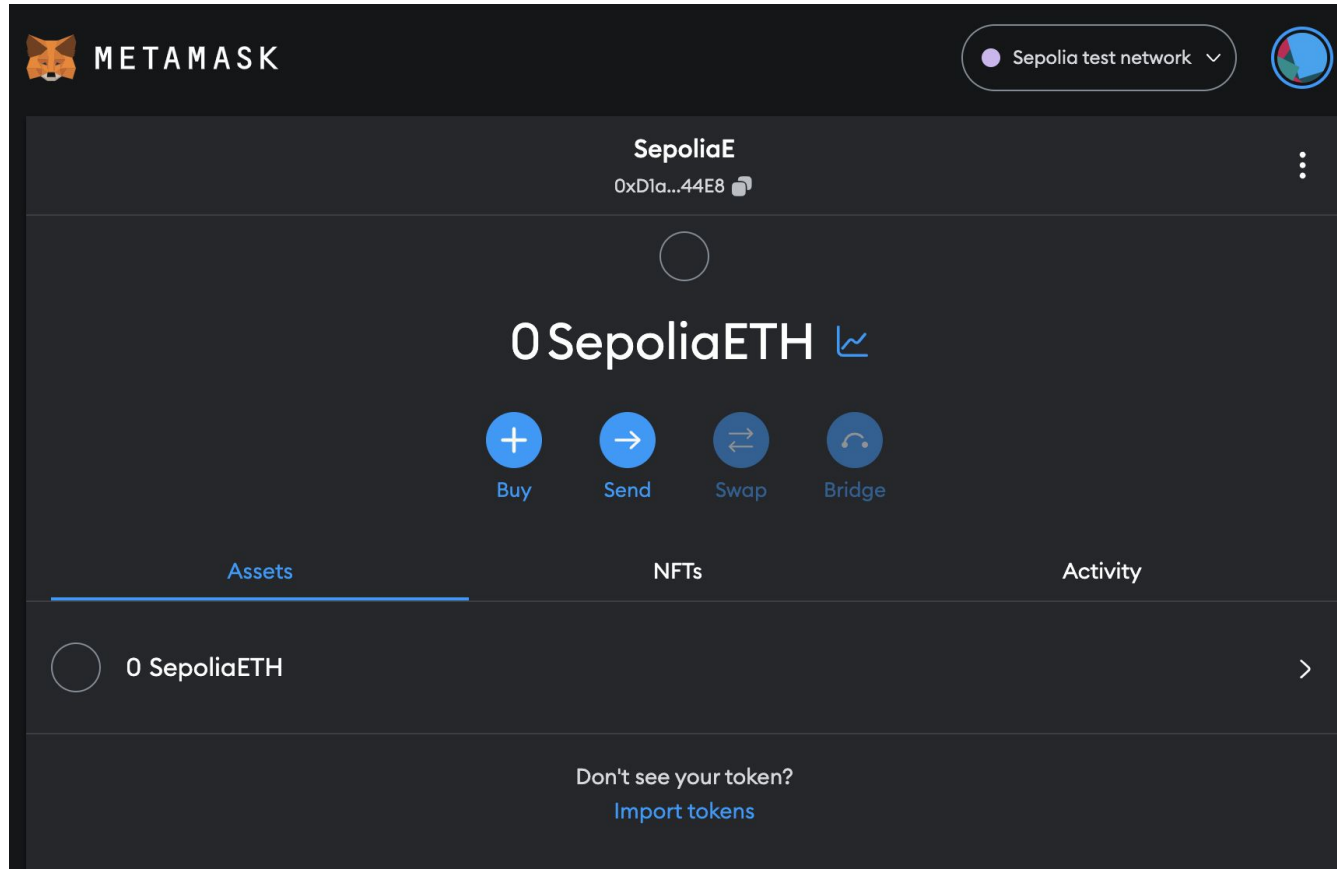


Execution Layer

Consensus Layer

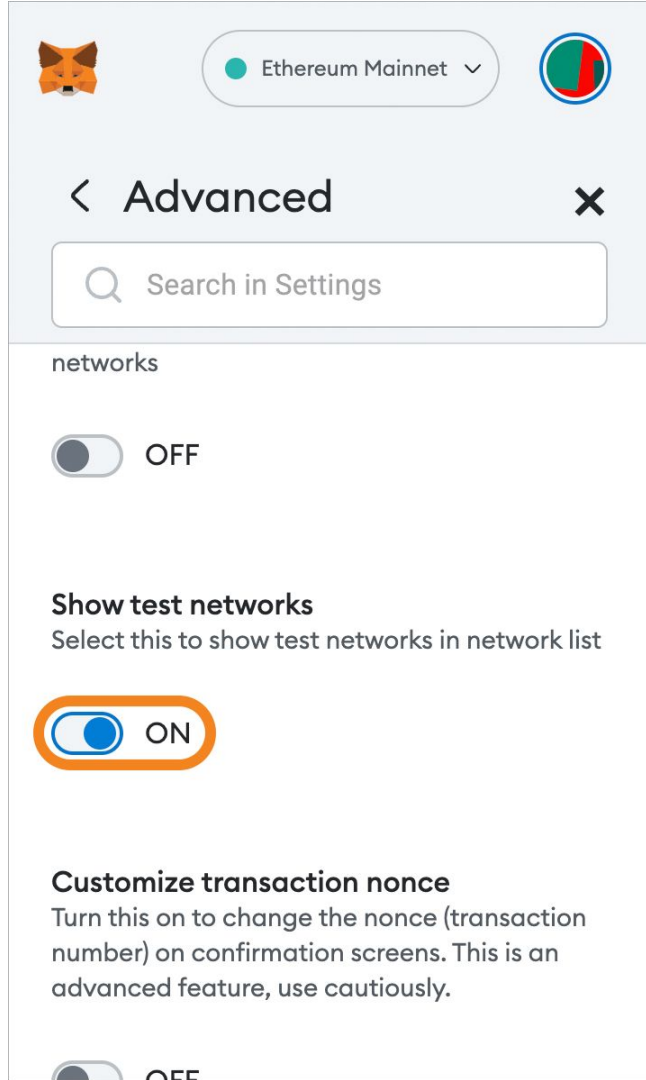


Wallet - METAMASK



Wallet - Testnet and Faucet

- Cambiar a una testnet
- Crear una nueva cuenta para obtener una dirección:
<https://support.metamask.io/hc/en-us/articles/13946422437147-How-to-view-testnets-in-MetaMask>
- Sepolia
<https://sepolia-faucet.pk910.de/>
-



Transferencia en Testnet

Smart contract

- Es un software donde la palabra contrat (contrato) no tienen ningún significado.
- Inmutable.
- Deterministico (contexto de la llamada y el estado de la blockchain)
- EVM

Despliegue de contrato sencillo

```
// Version of Solidity compiler this program was written for
pragma solidity 0.6.4;

// Our first contract is a faucet!
contract Faucet {
    // Accept any incoming amount
    receive() external payable {}

    // Give out ether to anyone who asks
    function withdraw(uint withdraw_amount) public {
        // Limit withdrawal amount
        require(withdraw_amount <= 1000000000000000000); //100,000,000,000,000,000 wei
        // Send the amount to the address that requested it
        msg.sender.transfer(withdraw_amount);
    }
}
```



Remix

remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.6.4+commit.1dca32f3.js

DEPLOY & RUN TRANSACTIONS ✓ >

Sepolia (11155111) network

ACCOUNT +
0xD1a...c44E8 (0.45476777)  

GAS LIMIT
3000000

VALUE
0 Wei

CONTRACT (Compiled by Remix)
Faucet - Faucet.sol

Deploy

☐ Publish to IPFS

OR

At Address Load contract from Address

Transactions recorded 10 ⓘ >

Deployed Contracts

FAUCET AT 0XB55...5EB12 (BLOC)  

Balance: 0.01999999889 ETH

withdraw 10000000000000000 

Low level interactions ⓘ

CALLDATA

Transact

Home Faucet.sol X

```
1 // Version of Solidity compiler this program was written for
2 pragma solidity 0.6.4;
3
4 // Our first contract is a faucet!
5 contract Faucet {
6     //Accept any incoming amount
7     receive() external payable {}
8
9     //Give out ether to anyone who asks
10    function withdraw(uint withdraw_amount) public {
11        //Limit withdrawal amount
12        require(withdraw_amount <= 10000000000000000);
13
14        // Send the amount to the address that requested it
15        msg.sender.transfer(withdraw_amount);
16    }
17 }
```

0 ☐ listen on all transactions Search with transaction hash or address

transact to Faucet.withdraw pending ...

[view on etherscan](#)

[block:3406566 txIndex:14] from: 0x560...c51C2 to: Faucet.withdraw(uint256) 0xb55...5EB

transact to Faucet.withdraw pending ...

transact to Faucet.withdraw errored: Returned error: {"jsonrpc":"2.0","error":{"execution reverted"},"id":1}

transact to Faucet.withdraw pending ...

[view on etherscan](#)

[block:3406569 txIndex:13] from: 0x560...c51C2 to: Faucet.withdraw(uint256) 0xb55...5EB